TYPE SCRIPT · CORE EXECUTION ENGINE

Deployment Job Orchestrator

Deep dive: bagaimana `src/core` mengubah deployment intent menjadi DAG runtime, transisi state, retry, dan rollback.

graph resolution

concurrent scheduler

failure recovery

01 · CORE MAP

Core bukan satu loop besar

FACADE

DeploymentOrchestrator

membuat run

→

PLAN

buildApiJobGraph

validasi DAG

→

RESOLVE

resolveJobCase

runtime jobs

→

CONTROL

Scheduler

ready + running

→

ACTION

JobExecutor

attempt + timeout

rollback.ts
reverse cleanup setelah failure

timeout.ts
validasi dan konversi unit

OrchestratorStore
boundary persistence untuk snapshot dan transisi

- Setiap modul mempersempit tanggung jawab: `intent → plan → runnable state → attempt → recovery` .

Referensi: `src/core/deployment-orchestrator.ts:17–117` , `src/core/api-job-graph.ts:10–61` , `src/core/job-definition.ts:24–109` , `src/core/scheduler.ts:9–114` , `src/core/job-executor.ts:13–151`

02 · FACADE

Satu call mengubah case menjadi hasil run

INPUT

```ts
type JobCaseDefinition = {
  jobId: string
  defaults?: {
    config?: { maxTimeout?: number }
  }
  steps: Record<string, JobCaseStep>
}
```

Deployment intent · belum executable

↓ deployCase()

```ts
async deployCase(deploymentCase, jobRunId) {
  const maxTimeout = caseMaxTimeout(deploymentCase)
  const runId = await this.createJobRunFromCase(
    deploymentCase, jobRunId
  )
  return this.runJobRun(runId, maxTimeout)
}
```

OUTPUT

```ts
type JobRunResult = {
  jobRun: {
    id: string
    status: JobRunStatus
  }
  jobs: JobStepRunRecord[]
}
```

State final · siap dibaca UI

1 load stored definition

2 resolve menjadi runtime jobs

3 snapshot ke database

4 jalankan scheduler

Referensi facade: `src/core/deployment-orchestrator.ts:56–75, 87–108` ; tipe: `src/types.ts:80–85, 196–199`

03 · GRAPH INPUT

Step ID berubah menjadi graph terurut

SEBELUM · PILIHAN TEMPLATE

```
[
  "vm",
  "attach-acl",
  "acl-rule",
  "subnet",
  "acl-list",
  "vpc"
]
```

Hanya logical ID. Urutan input tidak dipercaya sebagai urutan eksekusi.

SESUDAH · TOPOLOGICAL PLAN

```
[
  { id: "vpc",         dependsOn: [] },
  { id: "subnet",      dependsOn: ["vpc"] },
  { id: "acl-list",    dependsOn: ["vpc"] },
  { id: "vm",          dependsOn: ["subnet"] },
  { id: "attach-acl", dependsOn: ["subnet", "acl-list"] },
  { id: "acl-rule",    dependsOn: ["acl-list"] }
]
```

Dependency selalu muncul lebih dulu daripada dependent.

selected IDs › registry metadata › DFS › ordered graph

Referensi: `src/core/api-job-graph.ts:10–23, 25–61`

04 · GRAPH VALIDATION

DFS membangun urutan sekaligus menjaga invariant

```
if (visited.has(stepId)) return
if (visiting.has(stepId))
  throw new Error(`cycle detected at: ${stepId}`)

visiting.add(stepId)
for (const dependencyId of step.dependsOn) {
  if (!selected.has(dependencyId)) throw missingDependency()
  visit(dependencyId)
}

visiting.delete(stepId)
visited.add(stepId)
ordered.push({
  id: stepId,
  dependsOn: [...step.dependsOn],
  execution: step.execution,
})
```

VISITING

acl-list

sedang membuka dependency

dependency selesai ↓

VISITED

vpc

aman, tidak perlu diproses ulang

post-order push ↓

ORDERED

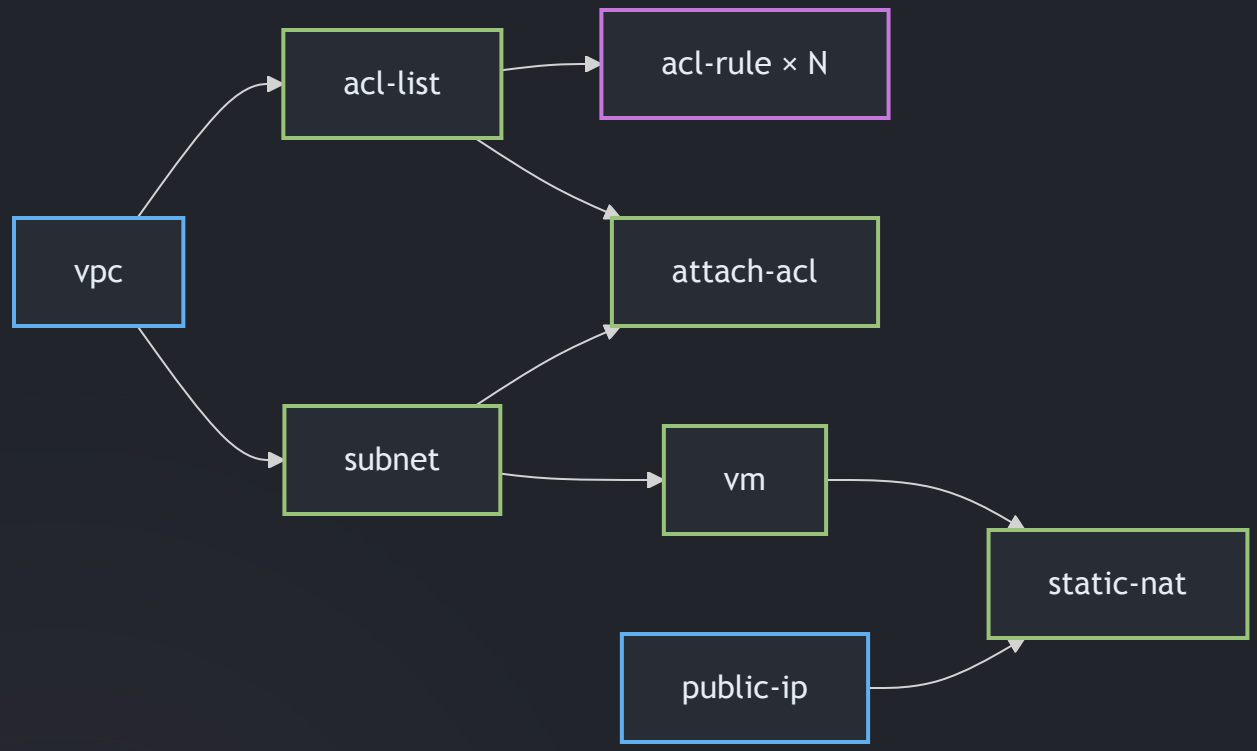
[vpc, acl-list]

dependency berada di depan

duplicate ID | unknown ID | missing dependency | cycle | fan-out dependent

Kode disederhanakan dari `src/core/api-job-graph.ts:14–60`

Dependency membuka paralelisme



`remaining = 0`

`vpc` dan `public-ip` langsung READY.

VPC sukses

`subnet` dan `acl-list` dibuka bersamaan.

Join

`static-nat` menunggu dua counter dependency menjadi nol.

Graph metadata: `src/requests/deployment-steps.ts:46–260` ; konsumsi graph: `src/core/scheduler.ts:22–35, 88–93`

06 · CASE RESOLUTION

Logical step berubah menjadi runtime job

SEBELUM

```
{
  "defaults": { "config": { "maxRetries": 4 } },
  "steps": {
    "acl-rule": {
      "instances": {
        "ssh":  { "input": { "startPort": 22 } },
        "http": { "input": { "startPort": 80 } }
      }
    }
  }
}
```

Input case: `src/interfaces/cli/cases/03.success-parallel-acl-rules.json:1–102`

SESUDAH

```
[
  {
    id: "acl-rule:ssh",
    type: "acl-rule",
    dependsOn: ["acl-list"],
    input: { startPort: 22 },
    maxRetries: 4
  },
  {
    id: "acl-rule:http",
    type: "acl-rule",
    dependsOn: ["acl-list"],
    input: { startPort: 80 },
    maxRetries: 4
  }
]
```

Resolver: `src/core/job-definition.ts:24–58, 62–109`

logical ID › runtime ID › independent state

07 · CONFIG PRECEDENCE

Override mengalir dari paling spesifik

1 · INSTANCE
maxRetries: 1
nilai paling spesifik

2 · STEP
maxRetries: 2
dipakai bila instance kosong

3 · CASE DEFAULT
maxRetries: 4
default per case

4 · ORCHESTRATOR
maxRetries: 0
fallback saat snapshot dibuat

```ts
const maxRetries = instance?.config?.maxRetries
  ?? configured.config?.maxRetries
  ?? deploymentCase.defaults?.config?.maxRetries

const maxTimeout = instance?.config?.maxTimeout
  ?? configured.config?.maxTimeout
  ?? deploymentCase.defaults?.config?.maxTimeout

return {
  id: runtimeId,
  type: step.type,
  dependsOn: [...step.dependsOn],
  ...(maxRetries === undefined ? {} : { maxRetries }),
}
```

HASIL UNTUK INSTANCE INI

`maxRetries = 1`

Core resolution: `src/core/job-definition.ts:62–109` ; orchestrator fallback: `src/core/deployment-orchestrator.ts:63–73`

Definition berubah menjadi indeks runtime

```
[
  { id: "vpc", dependsOn: [] },
  { id: "subnet", dependsOn: ["vpc"] },
  { id: "vm", dependsOn: ["subnet"] }
]
```

JobDefinition[] · bentuk persisten

↓ scheduler.run()

```
const definitionsById = new Map( ... )
const statuses = new Map( ... )
const dependents = new Map( ... )
const remaining = new Map( ... )
const ready: string[] = []
const running = new Map()
```

remaining

vpc → 0
subnet → 1
vm → 1

dependents

vpc → [subnet]
subnet → [vm]
vm → []

statuses

vpc → PENDING
subnet → PENDING
vm → PENDING

ready

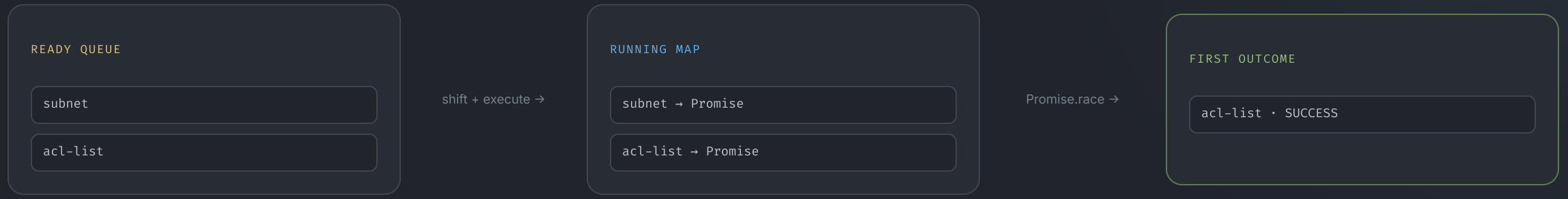
[vpc]

> `remaining` menjawab “berapa dependency belum sukses?” tanpa traversal graph berulang.

Referensi: `src/core/scheduler.ts:16–44, 63–67`

09 · CONCURRENCY LOOP

Ready queue berubah menjadi pekerjaan paralel




```
while (failedBy === undefined && ready.length > 0) {
  const jobId = ready.shift()!
  const controller = new AbortController()
  const result = executor.execute(jobRunId, jobId, controller.signal)
    .then(outcome => ({ jobId, outcome }))
  running.set(jobId, { controller, result })
}

const { jobId, outcome } = await Promise.race(
  [...running.values()].map(({ result }) => result)
)

for (const dependent of dependents.get(jobId)!) {
  const count = remaining.get(dependent)! - 1
  remaining.set(dependent, count)
  if (count === 0) await enqueue(dependent)
}
```

Incremental, bukan batch

Satu job yang selesai langsung membuka dependent-nya. Scheduler tidak menunggu semua job paralel dalam gelombang yang sama.

Referensi: `src/core/scheduler.ts:69–93`

Job persisten berubah menjadi handler context

1 · JOB STEP RECORD

```
{ jobId, type, attempt,
  input, apiControl }
```

store lookup →

2 · DEPENDENCY RESULTS

```
{ vpc: { id: "vpc-42" },
  aclList: { id: "acl-7" } }
```

runAttempt →

3 · JOB RUN CONTEXT

```
{ jobRunId, jobId, attempt,
  input, dependencyResults,
  signal, sleep }
```

```ts
return this.handlers[job.type]!.run({
  jobRunId: job.jobRunId,
  jobId: job.jobId,
  attempt: job.attempt,
  input: job.input,
  dependencyResults: await store.getDependencyResults(
    job.jobRunId, job.jobId
  ),
  signal: attemptSignal,
  sleep: ms => sleep(ms, attemptSignal),
})
```

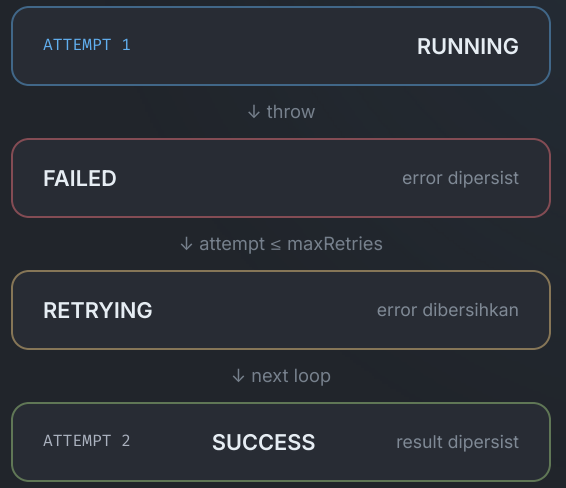
Core tidak mengenal CloudStack

Executor hanya memilih handler berdasarkan `type` dan memberi context generik. Implementasi domain berada di luar core.

Referensi: `src/core/job-executor.ts:107–125`; kontrak context: `src/types.ts:128–155`

Satu job bergerak lewat state yang persisten

```
while (true) {
  const current = await store.getJobStepRun(jobRunId, jobId)
  const attempt = current.attempt + 1
  const job = await store.transitionJob(jobRunId, jobId, {
    status: "RUNNING", attempt, error: null,
  })

  try {
    const result = await this.runAttempt(job, signal, timeoutMs)
    await store.transitionJob(jobRunId, jobId, {
      status: "SUCCESS", result, error: null,
    })
    return { success: true }
  } catch (error) {
    await store.transitionJob(jobRunId, jobId, {
      status: "FAILED", error: error.message,
    })
    if (signal.aborted || attempt > current.maxRetries)
      return { success: false, error }

    await store.transitionJob(jobRunId, jobId, {
      status: "RETRYING", error: null,
    })
  }
}
```



> `maxRetries = 1` berarti maksimal `2 attempts`.

Referensi: `src/core/job-executor.ts:21–51`

Dua sumber cancellation menjadi satu signal


```ts
const controller = new AbortController()
const signal = AbortSignal.any([externalSignal, controller.signal])

const timeout = new Promise<never>((_resolve, reject) => {
  timer = setTimeout(() => {
    const error = new Error(`Job ${jobId} timed out`)
    error.name = "TimeoutError"
    controller.abort(error)
    reject(error)
  }, timeoutMs)
})

try {
  return await Promise.race([operation(signal), timeout])
} finally {
  if (timer !== undefined) clearTimeout(timer)
}
```

> Timeout membatasi waktu menunggu attempt; tidak menjamin operasi remote sudah berhenti.

Referensi: `src/core/job-executor.ts:128–150` ; unit conversion: `src/core/timeout.ts:1–19`

Satu failure mengubah seluruh run

SEBELUM

vpc	SUCCESS
subnet	FAILED
acl-list	RUNNING
vm	PENDING

stopAfterFailure("subnet")
→

SESUDAH

run	FAILED
acl-list	ABORT SIGNAL
vm	SKIPPED
ready queue	EMPTY

```
failedBy = jobId
await store.setJobRunStatus(jobRunId, "FAILED")

for (const runningJob of running.values())
  runningJob.controller.abort(new Error(`stopped after ${jobId}`))

ready.length = 0
for (const [pendingId, status] of statuses) {
  if (["PENDING", "READY", "RETRYING"].includes(status))
    await store.transitionJob(jobRunId, pendingId, {
      status: "SKIPPED", error: `Blocked by ${jobId}`,
    })
}
```

Failure dikonsolidasikan sekali

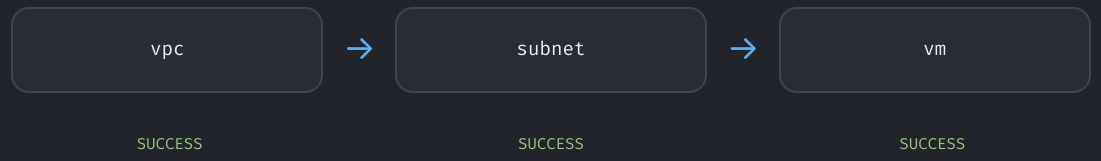
`failedBy` mencegah failure lain memulai koordinasi kedua saat beberapa promise selesai hampir bersamaan.

Referensi: `src/core/scheduler.ts:46–61, 94–96`

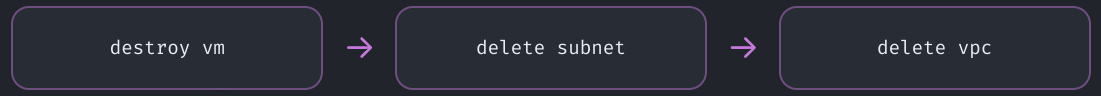
14 · ROLLBACK ORDER

Forward success berubah menjadi reverse cleanup

FORWARD · CREATE



REVERSE · CLEANUP

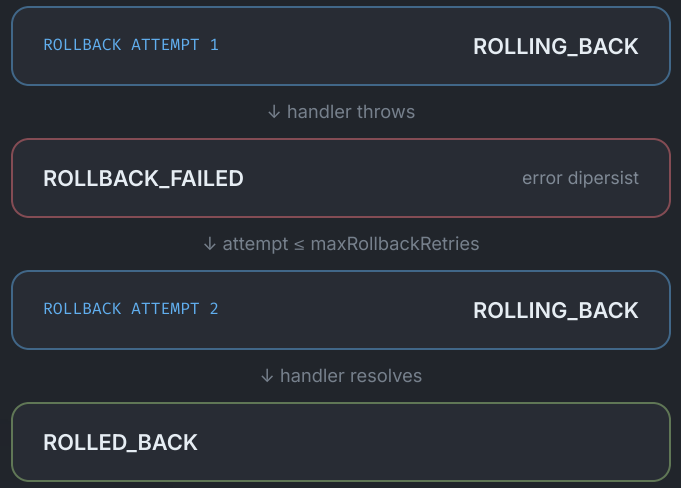


> Resource downstream dibersihkan sebelum dependency-nya.

```ts
await store.setJobRunStatus(jobRunId, "ROLLING_BACK")

const rollbackCandidates = new Set(
  (await store.getJobStepRuns(jobRunId))
    .filter(({ status }) =>
      ["SUCCESS", "ROLLING_BACK", "ROLLBACK_FAILED"].includes(status))
    .map(({ jobId }) => jobId)
)

let failed = false
for (const job of [...ordered].reverse()) {
  if (rollbackCandidates.has(job.id)
      && !(await executor.rollback(jobRunId, job.id))) {
    failed = true
  }
}

await store.setJobRunStatus(
  jobRunId, failed ? "ROLLBACK_FAILED" : "ROLLED_BACK"
)
```

Referensi: `src/core/rollback.ts:7–35`

Rollback punya lifecycle dan retry sendiri




```ts
const initial = await store.getJobStepRun(jobRunId, jobId)
const rollback = handlers[initial.type]?.rollback

if (rollback === undefined) {
  await transition("ROLLING_BACK")
  await transition("ROLLBACK_SKIPPED")
  return true
}

while (true) {
  const current = await store.getJobStepRun(jobRunId, jobId)
  const attempt = current.rollbackAttempt + 1
  const job = await transition("ROLLING_BACK")

  try {
    await withTimeout(() => rollback({
      attempt, input: job.input, result: job.result,
    }))
    await transition("ROLLED_BACK")
    return true
  } catch (error) {
    await transition("ROLLBACK_FAILED", error)
    if (attempt > current.maxRollbackRetries) return false
  }
}
```

> Attempt eksekusi dan rollback dihitung terpisah.

Kode disederhanakan dari `src/core/job-executor.ts:54–105`

16 · PERSISTENCE BOUNDARY

Core menulis event, store membangun state

SRC / CORE

transition intent

```
{ status: "RUNNING", attempt: 2 }
{ status: "FAILED", error: "timeout" }
{ status: "RETRYING", error: null }
{ status: "SUCCESS", result: { ... } }
```

transitionJob() →

← getJobStepRun()

DATABASE / STORE

append-only history

```
#12 · RUNNING  · attempt 2
#13 · FAILED   · timeout
#14 · RETRYING · null
#15 · SUCCESS  · result { ... }
```

ordered logs → group by jobId → latest state + timing → **JobStepRunRecord**

- Executor dan scheduler selalu membaca ulang state persisten; retry tidak hanya hidup di memory proses.

Call site core: `src/core/job-executor.ts:28–49`, `src/core/scheduler.ts:38–43`; implementasi store: `src/database/store.ts:210–272, 300–339`

17 · END-TO-END TRACE

Satu deployment melintasi tujuh transformasi

01
JobCaseDefinition
deployment intent

→

02
JobDefinition[]
resolved snapshot

→

03
Runtime indexes
remaining + dependents

→

04
Ready / Running
concurrent control

05
JobRunContext
handler boundary

→

06
Transition logs
persistent evidence

→

07
JobRunResult
final observable state

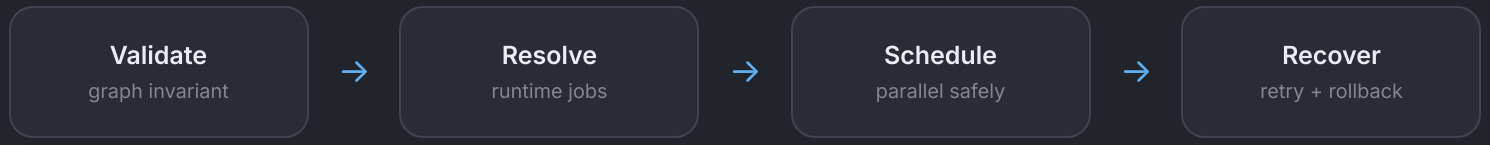
Facade	Graph	Resolve	Control
deployment-orchestrator.ts	api-job-graph.ts	job-definition.ts	scheduler.ts

Attempt	Recovery	Timeout
job-executor.ts	rollback.ts	timeout.ts

Seluruh modul utama: `src/core/`

TAKEAWAY

Core mengubah intent menjadi state yang dapat dipercaya



Graph memastikan urutan. Scheduler membuka paralelisme. Executor mengisolasi attempt. Persistence menjaga histori. Rollback mengembalikan konsistensi.